UNIVERSIDAD TECNOLÓGICA ECOTEC

Facultad de Ingenierías, Arquitectura y Ciencias de la Naturaleza

Ingeniería en Sistemas Inteligentes

PROGRAMACIÓN DE SISTEMAS WEB

Actividad Integradora 3

Sistema Web de Gestión de Estudiantes con PHP, PDO y MySQL

Informe técnico · Arquitectura MVC · Seguridad · Evidencias de ejecución

ESTUDIANTE	Carlos Julio Villegas Bravo
CARRERA	Ingeniería en Sistemas Inteligentes
ASIGNATURA	Programación de Sistemas Web
DOCENTE	Gregorio Cruz G. — Máster en Diseño y Gestión Web
FECHA	8 de noviembre de 2026

REPOSITORIO DEL PROYECTO

https://github.com/carlosvillegas2733-rgb/AI3_DisenoWeb_CarlosVillegas

Contenido

Informe técnico de la Actividad Integradora 3: análisis de la capa de acceso a datos, arquitectura MVC, medidas de seguridad, evidencias de ejecución y resultados de las pruebas.

1 **Resumen ejecutivo**
Objetivo, alcance, tecnologías y resultados obtenidos.

2 **Justificación técnica**
Comparación PDO vs MySQLi, decisión tecnológica y medidas de seguridad.

3 **Arquitectura del sistema**
Patrón MVC, diagrama de componentes, flujo de datos y estructura del proyecto.

4 **Evidencias de ejecución**
Capturas de la base de datos, código, formulario, validaciones, CRUD, diseño responsivo y commits.

5 **Pruebas y resultados**
Pruebas de sintaxis, unitarias, de integración HTTP y de seguridad.

6 **Enlaces, conclusiones y recomendaciones**
Repositorio, conclusiones, mejoras futuras y referencias.

A **Anexo A. Script de creación de la base de datos**
DDL, tipos de datos y restricciones de integridad.

B **Anexo B. Guía de despliegue local**
Requisitos, importación, configuración y ejecución.

Ficha del proyecto

Estudiante	Carlos Julio Villegas Bravo
Carrera	Ingeniería en Sistemas Inteligentes
Facultad	Facultad de Ingenierías, Arquitectura y Ciencias de la Naturaleza
Asignatura	Programación de Sistemas Web
Docente	Gregorio Cruz G. — Máster en Diseño y Gestión Web
Repositorio	`https://github.com/carlosvillegas2733-rgb/AI3_DisenoWeb_CarlosVillegas`
Stack	PHP 8.3, PDO, MySQL/MariaDB 11.4, HTML5, CSS3, JavaScript
Arquitectura	MVC modular con front controller y autoload PSR-4
Base de datos	`db_sistema_web` · tabla `estudiantes`
Pruebas	34 pruebas unitarias · 30 pruebas de integración HTTP
Fecha	8 de noviembre de 2026

1. Resumen ejecutivo

La Actividad Integradora 3 exige un módulo web full-stack que integre PHP con una base de datos relacional MySQL/MariaDB, aplicando una arquitectura MVC o modular, sentencias preparadas contra inyección SQL, saneamiento de salidas contra XSS, control de versiones en GitHub y documentación técnica en PDF. Este informe documenta la solución desarrollada y demuestra su funcionamiento con evidencias obtenidas de una ejecución real.

El sistema implementa la gestión académica de estudiantes: registro, consulta, búsqueda, edición y eliminación de registros. La aplicación se organizó en capas (controlador, modelo, validador y vistas) con un front controller en `public/index.php`. La conexión a datos se encapsuló en `app/Config/conexion.php` mediante PDO, con manejo de excepciones y mensajes genéricos hacia el usuario. Todas las consultas que reciben datos del formulario utilizan sentencias preparadas y todas las salidas HTML se escapan con `htmlspecialchars()`. Los formularios mutantes incluyen un token CSRF y las acciones destructivas solo aceptan POST.

La solución fue verificada con 34 pruebas unitarias sobre validación, repositorio y escape de HTML, 30 pruebas de integración HTTP sobre el flujo CRUD y los códigos de error, y una ejecución real en navegador con capturas de cada etapa. El repositorio público contiene diez commits atómicos de desarrollo, un `.gitignore` para PHP, un `README.md` con la guía de despliegue y este informe técnico.

10 columnas en la tabla principal	**34** pruebas unitarias aprobadas	**30** pruebas de integración HTTP
10 commits atómicos de desarrollo	**100 %** consultas con sentencias preparadas	**0** errores de sintaxis PHP/JS

Alcance y resultados

Funcionalidad entregada
CRUD completo de estudiantes, búsqueda por cédula, nombre, correo o carrera, validación de cédula ecuatoriana con dígito verificador, normalización de datos, confirmación de eliminación, mensajes flash y páginas de error 400/404/405/419/500.

Calidad técnica
Código PHP 8.3 con `declare(strict_types=1)`, estilo PSR-12, nombres semánticos, separación de responsabilidades, autoload PSR-4, pruebas automatizadas y documentación reproducible.

Seguridad
PDO con `ATTR_EMULATE_PREPARES = false`, sentencias preparadas, escape de salidas, CSRF con `hash_equals()`, validación servidor, cabeceras de seguridad y cookies `HttpOnly` y `SameSite=Lax`.

Verificación
Pruebas unitarias, integración HTTP, flujo real en navegador, prueba de XSS almacenado, prueba de CSRF sin token, verificación de métodos HTTP y validación de sintaxis de todos los archivos.

2. Justificación técnica

2.1 Análisis comparativo entre PDO y MySQLi

PHP ofrece dos extensiones principales para acceder a MySQL/MariaDB: **MySQLi** y **PDO** (PHP Data Objects). Ambas son oficiales, están mantenidas y permiten usar sentencias preparadas. La diferencia no está en si pueden conectarse a MySQL, sino en el nivel de abstracción, la portabilidad, la ergonomía del manejo de errores y el acoplamiento al motor.

MySQLi es una extensión específica para MySQL/MariaDB. Expone funciones y clases propias del motor, como consultas asíncronas, `mysqli_result`, `mysqli_stmt` y operaciones múltiples. Su API existe en dos estilos (orientado a objetos y procedural), lo que puede generar inconsistencias en un equipo. **PDO**, en cambio, es una capa de abstracción de acceso a datos: el programador usa una API uniforme y el driver correspondiente se selecciona mediante el DSN. Cambiar de MySQL a PostgreSQL o SQLite implica cambiar el DSN y el driver, no reescribir la lógica de acceso a datos.

En rendimiento, ambas extensiones usan el protocolo nativo de MySQL y pueden ejecutar sentencias preparadas del lado del servidor. PDO añade una capa de abstracción mínima; en microbenchmarks MySQLi puede ser ligeramente más directo, pero en un CRUD académico con decenas o cientos de registros la diferencia es imperceptible frente a los beneficios de mantenibilidad, portabilidad y manejo de excepciones. La decisión técnica debe priorizar el costo total de propiedad y la claridad del código.

En seguridad, ambas soportan sentencias preparadas. La diferencia práctica es que PDO permite `PDO::ATTR_ERRMODE = PDO::ERRMODE_EXCEPTION`, lo que integra los errores de base de datos con el mecanismo de excepciones de PHP y facilita un `try-catch` centralizado. PDO también admite parámetros nombrados (`:cedula`, `:correo`), que reducen el riesgo de confundir el orden de los valores en consultas largas. MySQLi usa únicamente marcadores posicionales (`?`) y, por defecto, su modo silencioso obliga a comprobar manualmente cada retorno o a activar `mysqli_report()`.

Criterio	PDO	MySQLi	Valoración para el proyecto
Naturaleza	Capa de abstracción con drivers	Extensión específica de MySQL/MariaDB	PDO ofrece una API más general y desacoplada.
Portabilidad	Cambiar el DSN permite cambiar de motor	Requiere reescribir la capa de datos	PDO facilita una futura migración.
Rendimiento	Capa mínima; prepares nativos al desactivar emulación	Acceso directo al motor	Diferencia irrelevante para el volumen del módulo.
Sentencias preparadas	Nativas o emuladas; se configuran explícitamente	Nativas con `bind_param()`	PDO con `ATTR_EMULATE_PREPARES = false` cumple el requisito.
Parámetros nombrados	Sí (`:nombre`)	No (solo `?`)	PDO mejora la legibilidad y reduce errores.
Manejo de errores	Excepciones con `PDOException`	Modo silencioso por defecto; requiere configuración	PDO se integra mejor con `try-catch`.

Criterio	PDO	MySQLi	Valoración para el proyecto
API	Orientada a objetos y uniforme	Orientada a objetos y procedural	PDO es más consistente para el equipo.
Ecosistema	Usado por frameworks y librerías modernas	Común en código legado y utilidades específicas	PDO tiene mejor proyección.

2.2 Decisión tecnológica y configuración de la conexión

Se seleccionó **PDO** por cuatro razones: portabilidad, manejo de excepciones, parámetros nombrados y una API uniforme. El proyecto no necesita características exclusivas de MySQLi (consultas asíncronas o múltiples), por lo que el costo de la abstracción es bajo y el beneficio es alto. La conexión se aisló en `app/Config/conexion.php`, se configuró con `charset=utf8mb4` y se protegieron las credenciales con variables de entorno y valores por defecto para el entorno local.

app/Config/conexion.php · configuración de PDO

```
$dsn = sprintf(
    'mysql:host=%s;port=%s;dbname=%s;charset=%s',
    $config['host'],
    $config['port'],
    $config['database'],
    $config['charset']
);

try {
    $pdo = new PDO($dsn, $config['username'], $config['password'], [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
} catch (PDOException $exception) {
    error_log('[database] ' . $exception->getMessage());
    throw new RuntimeException(
        'No se pudo establecer la conexión con la base de datos.',
        0,
        $exception
    );
}
```

Cada opción cumple una función concreta: `ERRMODE_EXCEPTION` convierte los errores de la base de datos en excepciones capturables; `FETCH_ASSOC` devuelve arreglos asociativos que las vistas consumen de forma explícita; `ATTR_EMULATE_PREPARES = false` obliga a usar sentencias preparadas reales del servidor; y `charset=utf8mb4` garantiza el soporte completo de Unicode (acentos, emojis y caracteres especiales). El mensaje genérico evita exponer credenciales, nombres de tablas o trazas internas al usuario.

2.3 Medidas de seguridad implementadas

Sentencias preparadas contra inyección SQL

El repositorio `app/Models/Estudiante.php` usa `prepare()` y `execute()` en todas las operaciones que reciben datos del usuario. La consulta se envía como una plantilla con marcadores; los valores viajan por un canal

separado y la base de datos los trata como datos, no como fragmentos de SQL. Una entrada como `' OR '1'='1` se convierte en un valor literal y no altera la estructura de la consulta.

app/Models/Estudiante.php · inserción con sentencia preparada

```
$statement = $this->pdo->prepare(
    'INSERT INTO estudiantes (cedula, nombres, apellidos, correo, carrera, semestre, telefono)
     VALUES (:cedula, :nombres, :apellidos, :correo, :carrera, :semestre, :telefono)'
);
$statement->execute($this->bindings($data));
```

La búsqueda también usa sentencias preparadas, pero con marcadores posicionales y la función `LOCATE()` en lugar de concatenar comodines. Así se evita la inyección por comodines y se mantiene la legibilidad:

app/Models/Estudiante.php · búsqueda parametrizada

```
$sql = 'SELECT id, cedula, nombres, apellidos, correo, carrera, semestre, telefono, fecha_creacion
        FROM estudiantes';
$params = [];

if ($search !== '') {
    $sql .= ' WHERE LOCATE(?, cedula) > 0
                OR LOCATE(?, nombres) > 0
                OR LOCATE(?, apellidos) > 0
                OR LOCATE(?, correo) > 0
                OR LOCATE(?, carrera) > 0';
    $params = array_fill(0, 5, $search);
}

$sql .= ' ORDER BY apellidos ASC, nombres ASC';
$statement = $this->pdo->prepare($sql);
$statement->execute($params);
```

Saneamiento de salidas contra XSS

Todas las salidas dinámicas pasan por la función `e()`, que delega en `htmlspecialchars()` con `ENT_QUOTES` (escapa comillas simples y dobles), `ENT_SUBSTITUTE` (reemplaza secuencias UTF-8 inválidas) y codificación UTF-8. Esto impide que un valor almacenado como `<script>alert("xss")</script>` se interprete como código en el navegador: se muestra como texto y no se ejecuta.

app/Core/helpers.php · escape de salidas

```
function e(?string $value): string
{
    return htmlspecialchars(
        (string) $value,
        ENT_QUOTES | ENT_SUBSTITUTE,
        'UTF-8'
    );
}
```

La prueba automatizada `Escapes HTML output to prevent XSS` verifica que la entrada `<script>alert(1)</script>` se transforme en `<script>alert(1)</script>`. Además, la prueba de integración inserta un registro con una carga XSS en la base de datos y confirma que el listado la muestra escapada, sin ejecutarla.

Protección CSRF

Cada sesión genera un token aleatorio de 32 bytes con `random_bytes()`. El token se incluye en un campo oculto de los formularios y se valida con `hash_equals()`, una comparación de tiempo constante que evita ataques de temporización. La validación se aplica en todas las acciones mutantes (`guardar`, `actualizar`, `eliminar`). Si el token falta o no coincide, el sistema responde con HTTP 419 y una página de sesión expirada.

app/Core/Csrf.php · generación y validación

```
public static function token(): string
{
    if (!isset($_SESSION[self::SESSION_KEY]) || !is_string($_SESSION[self::SESSION_KEY])) {
        $_SESSION[self::SESSION_KEY] = bin2hex(random_bytes(32));
    }

    return $_SESSION[self::SESSION_KEY];
}

public static function validate(?string $token): bool
{
    return is_string($token)
        && isset($_SESSION[self::SESSION_KEY])
        && is_string($_SESSION[self::SESSION_KEY])
        && hash_equals($_SESSION[self::SESSION_KEY], $token);
}
```

Validación y normalización de datos

La validación del servidor es la frontera de seguridad; la validación del cliente solo mejora la experiencia. El validador `EstudianteValidator` aplica reglas explícitas y devuelve mensajes en español por campo:

- **Cédula:** exactamente 10 dígitos, provincia válida, tercer dígito de persona natural y dígito verificador calculado con el algoritmo ecuatoriano (coeficientes 2,1,2,1,2,1,2,1,2).
- **Nombres y apellidos:** entre 2 y 80 caracteres, solo letras Unicode, espacios, apóstrofes, puntos y guiones; se colapsan espacios repetidos.
- **Correo:** obligatorio, máximo 120 caracteres, validado con `filter_var(FILTER_VALIDATE_EMAIL)` y normalizado a minúsculas.
- **Carrera:** obligatoria y restringida a una lista blanca de programas de ECOTEC.
- **Semestre:** entero entre 1 y 8.
- **Teléfono:** entre 7 y 15 dígitos, con `+` opcional; se eliminan espacios, guiones y paréntesis antes de persistir.

Manejo de errores, sesión y cabeceras

El front controller captura `HttpException` y cualquier `Throwable`. Los detalles se registran con `error_log()` y el usuario recibe una página genérica. Las sesiones usan cookies `HttpOnly` y `SameSite=Lax`; las respuestas incluyen `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` y `Referrer-Policy: strict-origin-when-cross-origin`. Las acciones destructivas solo aceptan POST y los identificadores se validan con `filter_var()`.

Integridad en la base de datos

La tabla `estudiantes` declara clave primaria autoincremental, restricciones `UNIQUE` para cédula y correo, un `CHECK` para el semestre, índices para las búsquedas y marcas de auditoría `fecha_creacion` y `fecha_actualizacion`. El motor InnoDB y el conjunto `utf8mb4_unicode_ci` garantizan transacciones, integridad referencial y soporte Unicode.

Riesgo	Medida implementada	Evidencia
Inyección SQL	PDO + sentencias preparadas + `ATTR_EMULATE_PREPARES=false`	Figuras 2 y 3; pruebas del repositorio
XSS reflejado/almacenado	`htmlspecialchars()` con `ENT_QUOTES` y `ENT_SUBSTITUTE`	Figura 7; prueba de escape y prueba de XSS almacenado
CSRF	Token aleatorio por sesión + `hash_equals()` + POST obligatorio	Figura 8; prueba de POST sin token (419)
Datos inválidos	Validación servidor, normalización, listas blancas y dígito verificador	Figuras 5 y 6; pruebas unitarias
Exposición de errores	`try-catch`, logging interno y páginas genéricas	Sección 2.3; pruebas 400/404/405/419
Secuestro de sesión	Cookie `HttpOnly` y `SameSite=Lax`	`app/bootstrap.php`
Clickjacking / MIME sniffing	Cabeceras `X-Frame-Options` y `X-Content-Type-Options`	`public/index.php`

Defensa en profundidad. Ninguna medida aislada es suficiente. La aplicación combina validación de entrada, parametrización de consultas, escape de salida, tokens CSRF, restricciones de base de datos y manejo controlado de errores para reducir la superficie de ataque en cada capa.

3. Arquitectura del sistema

3.1 Patrón MVC y organización modular

La aplicación implementa una variante ligera del patrón Modelo-Vista-Controlador, sin framework y sin dependencias externas de Composer. Esta decisión permite que el evaluador vea directamente cómo se separan las responsabilidades y evita que la herramienta oculte la lógica que la asignatura busca evaluar. El autoload sigue la convención PSR-4: el prefijo `App\` se mapea al directorio `app/`.

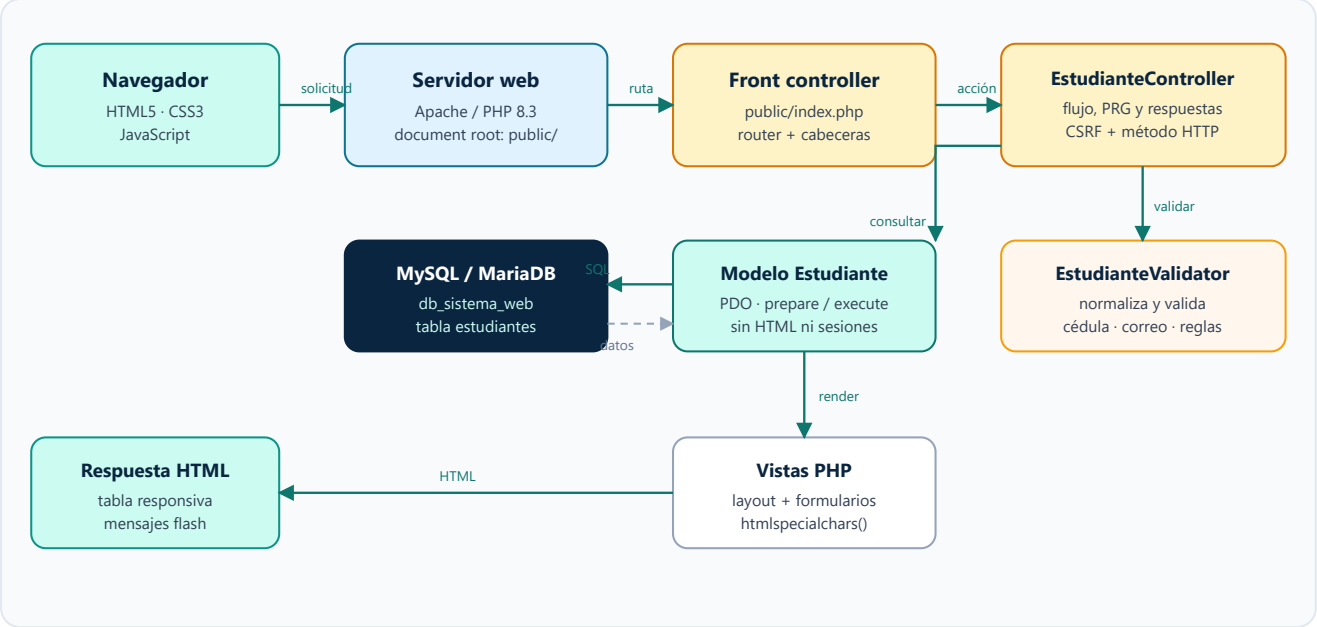
- **Modelo:** `App\Models\Estudiante` encapsula el acceso a datos. No conoce HTML ni sesiones; recibe y devuelve arreglos.
- **Vista:** archivos PHP en `app/Views/` que reciben datos ya preparados y escapan cada salida. No ejecutan SQL.
- **Controlador:** `App\Controllers\EstudianteController` interpreta la petición, valida, aplica el patrón PRG y elige la vista o la redirección.
- **Núcleo:** `Csrf`, `Flash`, `View`, `HttpException` y helpers compartidos.
- **Configuración:** `app/Config/conexion.php` centraliza el DSN y las credenciales.
- **Front controller:** `public/index.php` aplica cabeceras, resuelve la acción y captura errores.

Estructura del proyecto

```
AI3_DisenoWeb_CarlosVillegas/
├── app/
│   ├── Config/conexion.php
│   ├── Controllers/EstudianteController.php
│   ├── Core/{Csrf, Flash, HttpException, View, helpers}.php
│   ├── Models/Estudiante.php
│   ├── Validators/EstudianteValidator.php
│   ├── Views/{layout, estudiantes, errors}/
│   └── bootstrap.php
├── database/db_sistema_web.sql
├── docs/
│   ├── AI3_Informe_CarlosVillegas.pdf
│   └── evidencias/
├── public/
│   ├── assets/css/app.css
│   ├── assets/js/app.js
│   └── index.php
├── tests/run.php
├── README.md
├── .editorconfig
├── .gitattributes
└── .gitignore
```

3.2 Diagrama de arquitectura y flujo de datos

El diagrama muestra el recorrido de una petición desde el navegador hasta la base de datos y el regreso de la respuesta. Las flechas continuas representan el flujo de la solicitud; las discontinuas, el retorno de datos o resultados. El controlador es el único componente que coordina validación, modelo y vista; el modelo es el único que habla con la base de datos; la vista es la única que produce HTML.



3.3 Flujo de una operación de registro

1. El usuario completa el formulario y el navegador envía un `POST` a `index.php?accion=guardar`.
2. El front controller aplica cabeceras de seguridad y delega en `EstudianteController::store()`.
3. El controlador verifica que el método sea POST y valida el token CSRF; si falla, responde 419.
4. El validador normaliza los datos (cédula, correo, teléfono, espacios) y aplica las reglas de formato.
5. El repositorio comprueba la unicidad de cédula y correo y ejecuta un `INSERT` preparado.
6. Si hay errores, se vuelve a renderizar el formulario con los datos ingresados y los mensajes por campo.
7. Si la inserción es correcta, se guarda un mensaje flash, se responde con `302` y se redirige al listado (patrón PRG).
8. El listado recupera los registros con una consulta preparada y los muestra escapados con `htmlspecialchars()`.

> **Patrón PRG (Post/Redirect/Get).** Después de una mutación exitosa, el servidor redirige en lugar de renderizar directamente el resultado. Esto evita que el usuario reenvíe el formulario al actualizar la página y mantiene una única representación de la URL del listado.

4. Evidencias de ejecución

Las siguientes capturas fueron obtenidas de la ejecución real del sistema contra MariaDB 11.4.8 y del repositorio público en GitHub. Cada figura incluye una interpretación de lo que demuestra y por qué es relevante para los objetivos de la actividad.

4.1 Estructura de la base de datos

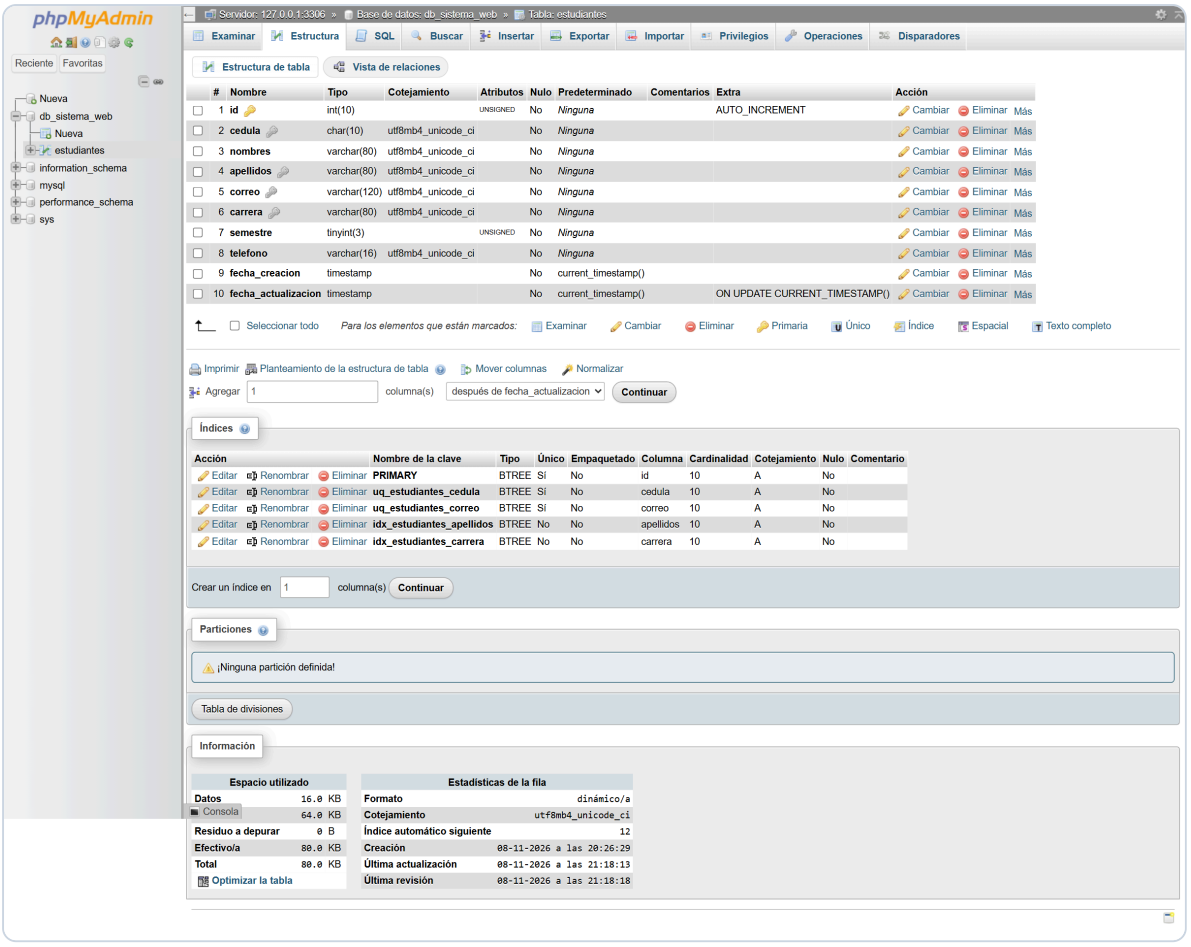


Figura 1. Estructura de la tabla `estudiantes` en phpMyAdmin. **Evidencia obligatoria**
Fuente: ejecución local del sistema, base de datos `db_sistema_web`, 2026.

Interpretación. La captura muestra las diez columnas de la tabla principal: `id` como clave primaria autoincremental, `cedula` y `correo` con restricciones `UNIQUE`, nombres y apellidos con longitud controlada, carrera, semestre, teléfono y las dos marcas de auditoría. También se observan los índices `uq_estudiantes_cedula`, `uq_estudiantes_correo`, `idx_estudiantes_apellidos` e `idx_estudiantes_carrera`, el motor InnoDB y el conjunto `utf8mb4_unicode_ci`. Esto demuestra que el diseño cumple el mínimo de seis campos, incluye auditoría y aplica integridad a nivel de motor, no solo en el código.

Normalización. Los campos son atómicos (1FN), la clave primaria es simple y todos los atributos dependen de ella (2FN), y no existen dependencias transitivas entre atributos no clave (3FN). La carrera se almacena como etiqueta validada por una lista blanca; si el catálogo creciera, el siguiente paso sería una tabla `carreras` con una clave foránea. Para el alcance de la actividad, una sola tabla principal con datos coherentes es la solución adecuada.

4.2 Sentencias preparadas y conexión PDO

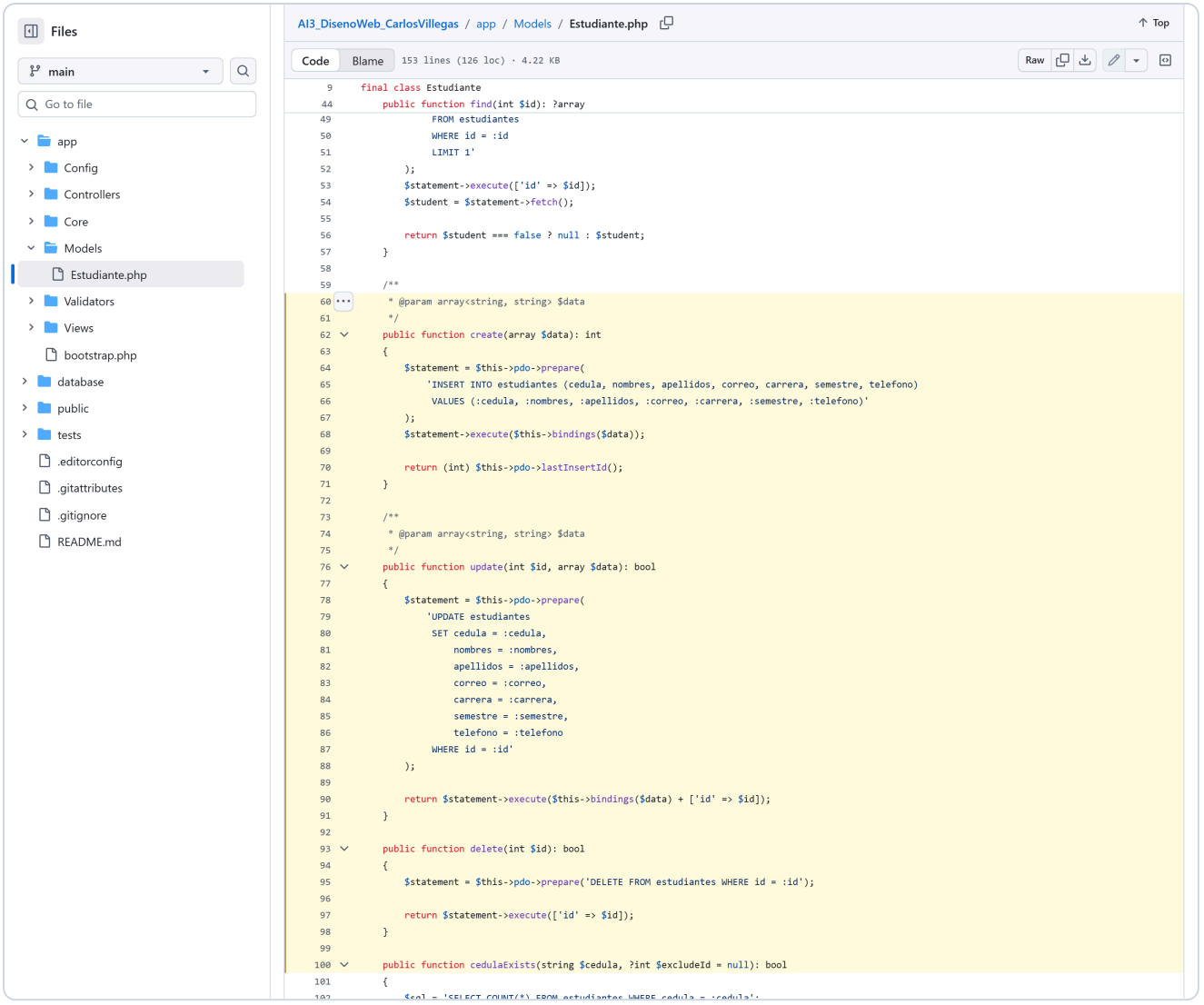


Figura 2. Código fuente con `prepare()` y `execute()` en el repositorio. Evidencia obligatoria
Fuente: repositorio público, archivo `app/Models/Estudiante.php`, 2026.

Interpretación. La captura muestra el código real del repositorio, con resaltado de sintaxis y números de línea. Se observan las operaciones de inserción, actualización y eliminación construidas con `prepare()` y ejecutadas con `execute()`; no hay concatenación de variables en las cadenas SQL. Los marcadores nombrados (`:cedula`, `:correo`, etc.) separan la estructura de la consulta de los valores. Esta es la evidencia directa de que la aplicación usa sentencias preparadas en la ruta crítica de escritura.

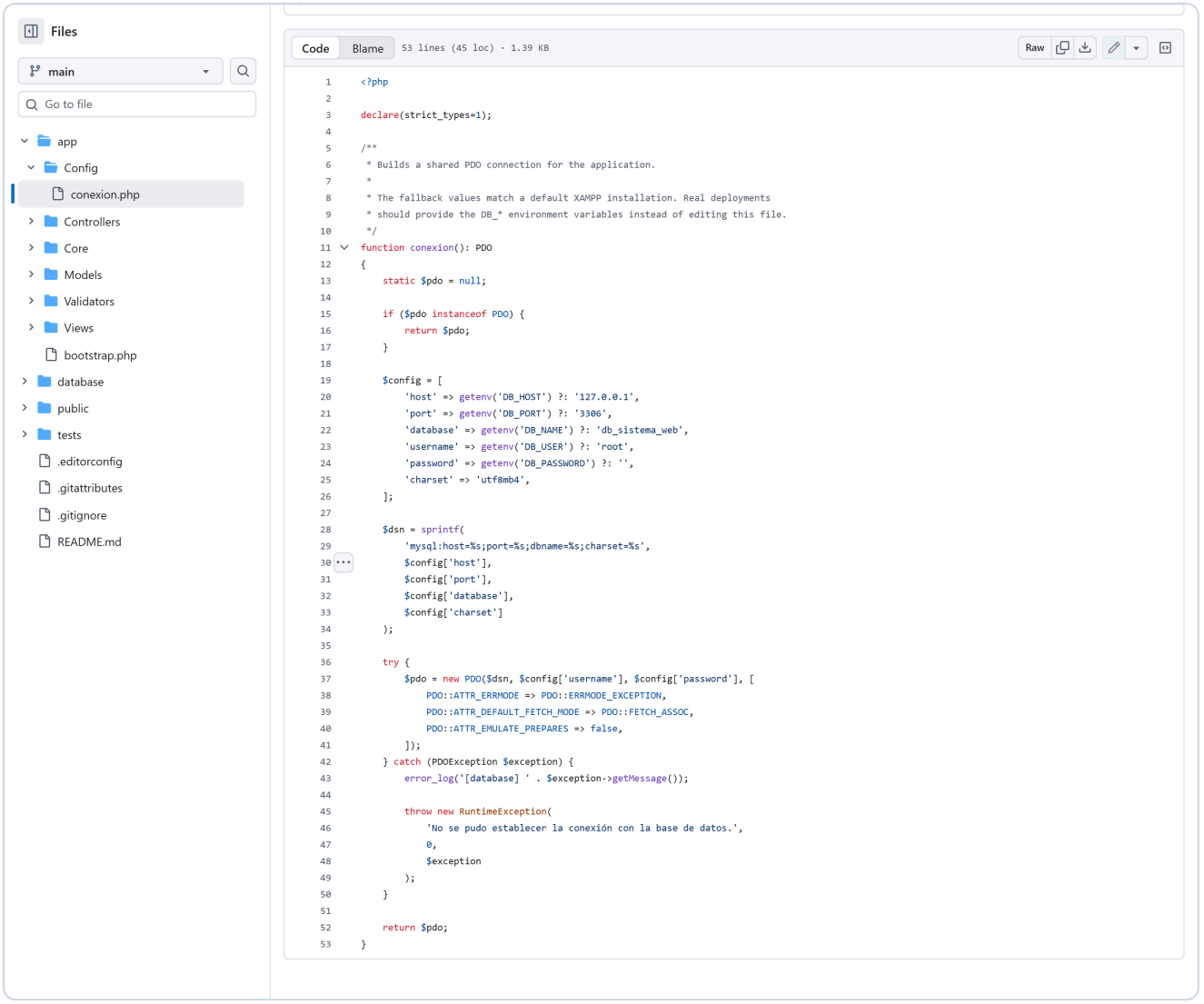


Figura 3. Conexión PDO con `ERRMODE_EXCEPTION` y `ATTR_EMULATE_PREPARES=false`.
Fuente: repositorio público, archivo `app/Config/conexion.php`, 2026.

Interpretación. La conexión se crea una sola vez y se reutiliza; el DSN incluye `charset=utf8mb4` y las opciones configuran excepciones, modo de obtención asociativo y sentencias preparadas nativas. El bloque `try-catch` registra el detalle técnico en el log y lanza una excepción genérica, de modo que el usuario nunca ve credenciales ni trazas internas. Esta separación entre configuración, error técnico y mensaje de usuario es una práctica de producción que el informe documenta explícitamente.

4.3 Formulario de registro

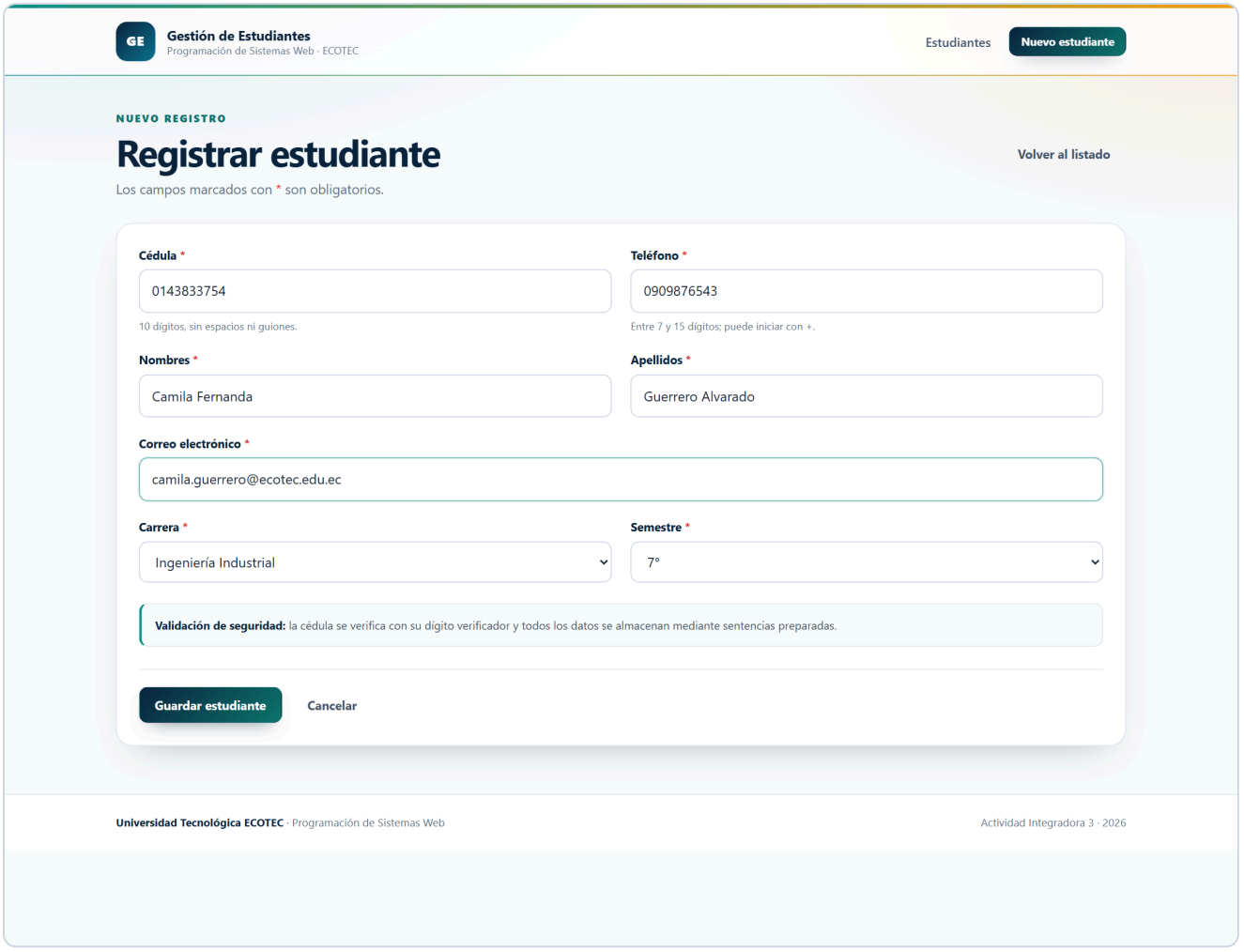


Figura 4. Formulario de registro con datos de prueba. Evidencia obligatoria
Fuente: ejecución local del sistema, 2026.

Interpretación. El formulario usa etiquetas asociadas, tipos HTML5 (`email`, `tel`, `select`), atributos `required`, `maxlength`, `inputmode` y `autocomplete`. Incluye un campo oculto con el token CSRF, textos de ayuda y una nota que explica la validación de la cédula y el uso de sentencias preparadas. Los datos de prueba corresponden a una cédula ecuatoriana válida y a un correo institucional; el formulario está listo para enviarse.

4.4 Validación en cliente y servidor

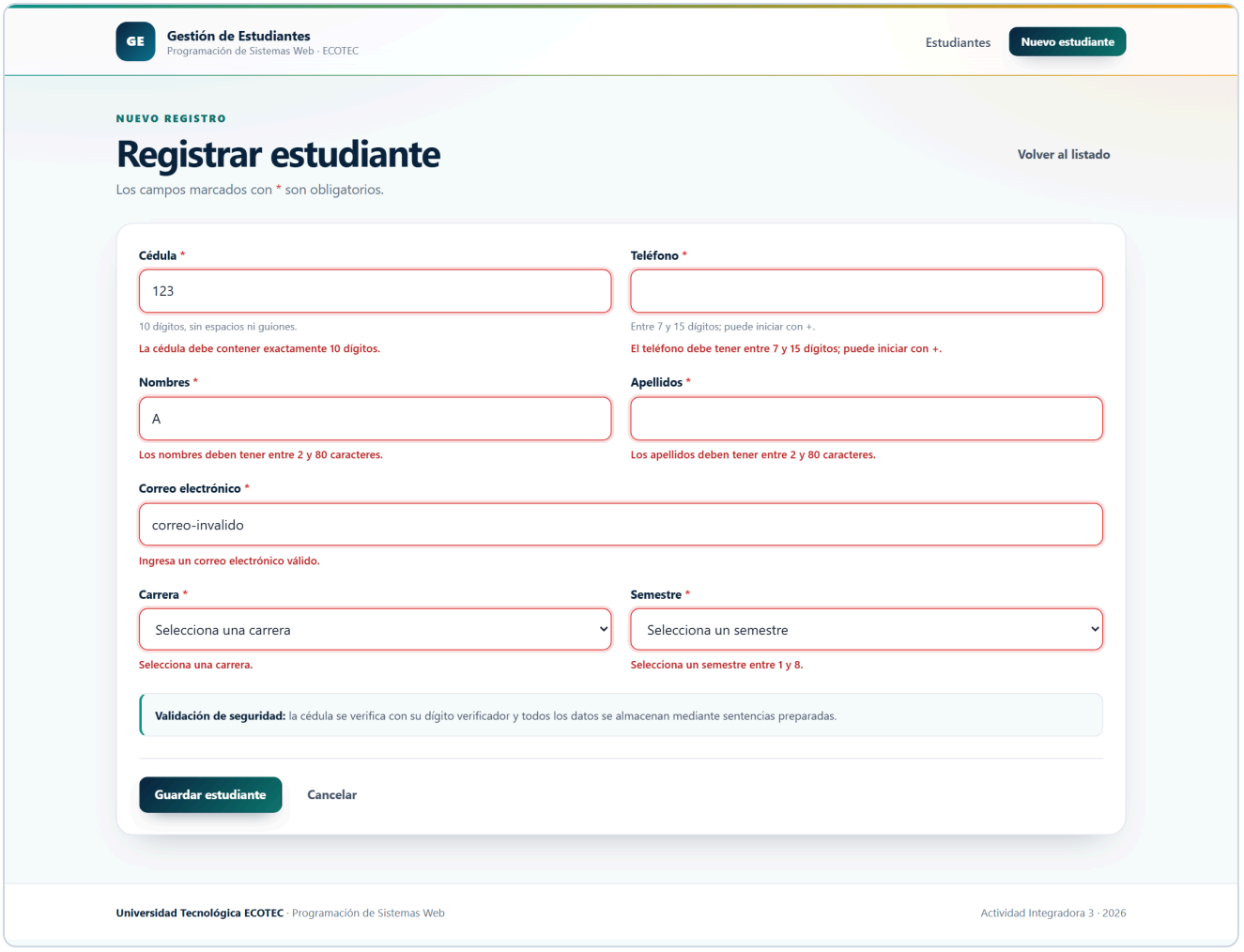


Figura 5. Validación en el cliente: errores por campo y primer campo inválido enfocado.
Fuente: ejecución local del sistema, 2026.

Interpretación. Al intentar enviar un formulario incompleto, el JavaScript recorre todos los campos, marca los inválidos con borde rojo y muestra un mensaje específico debajo de cada uno. La página no se recarga y el foco se coloca en la cédula, el primer campo inválido. Esta validación mejora la experiencia, pero no sustituye la validación del servidor: es una primera capa de defensa.

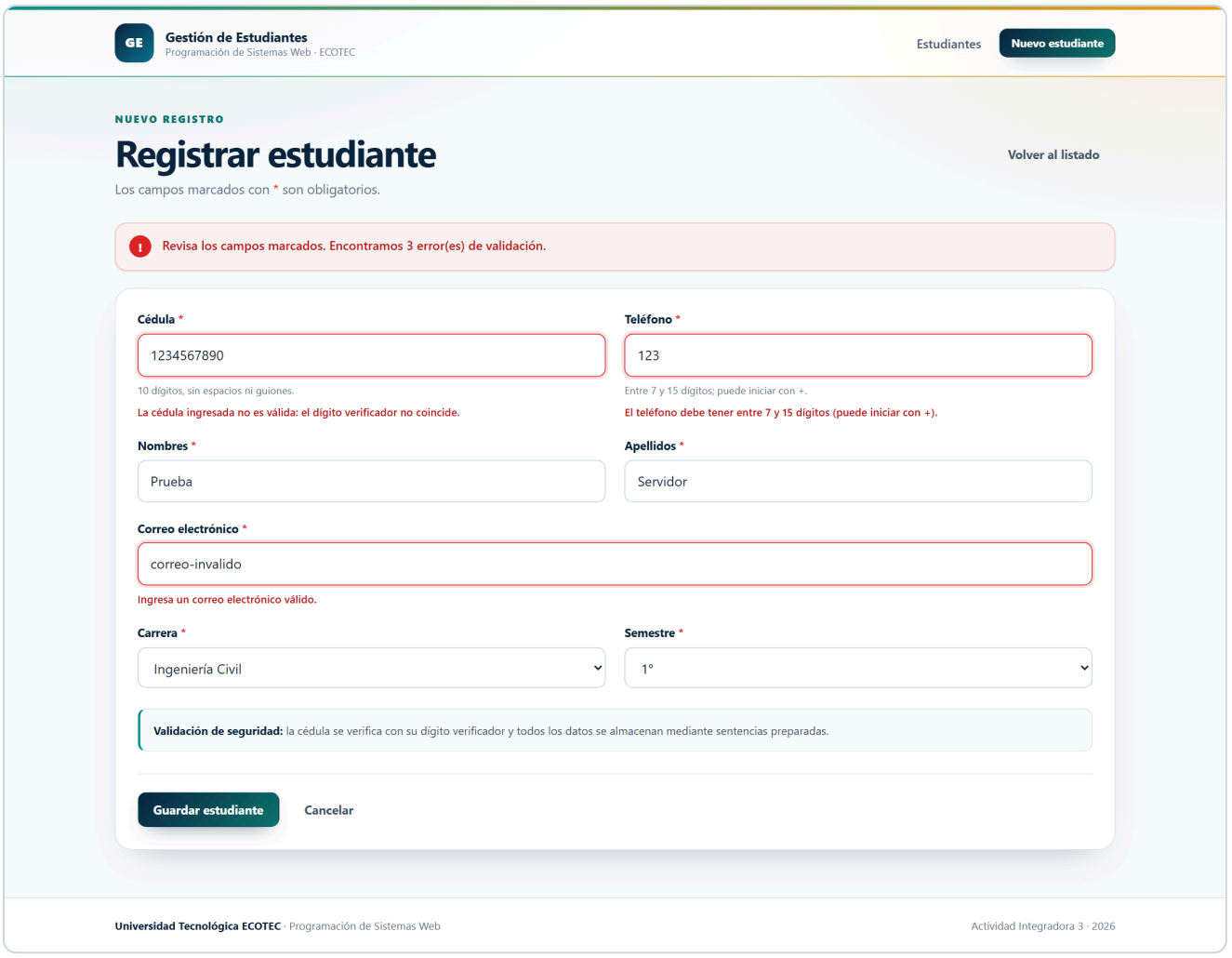


Figura 6. Validación en el servidor con JavaScript deshabilitado.
Fuente: ejecución local del sistema, 2026.

Interpretación. Con JavaScript deshabilitado, el formulario se envía igualmente y el servidor rechaza la cédula con dígito verificador incorrecto, el correo inválido y el teléfono fuera de rango. El resumen superior indica tres errores y cada campo conserva su mensaje. Esto demuestra que la seguridad no depende del navegador y que el servidor aplica las reglas incluso ante un cliente manipulado.

4.5 Consulta, búsqueda y visualización

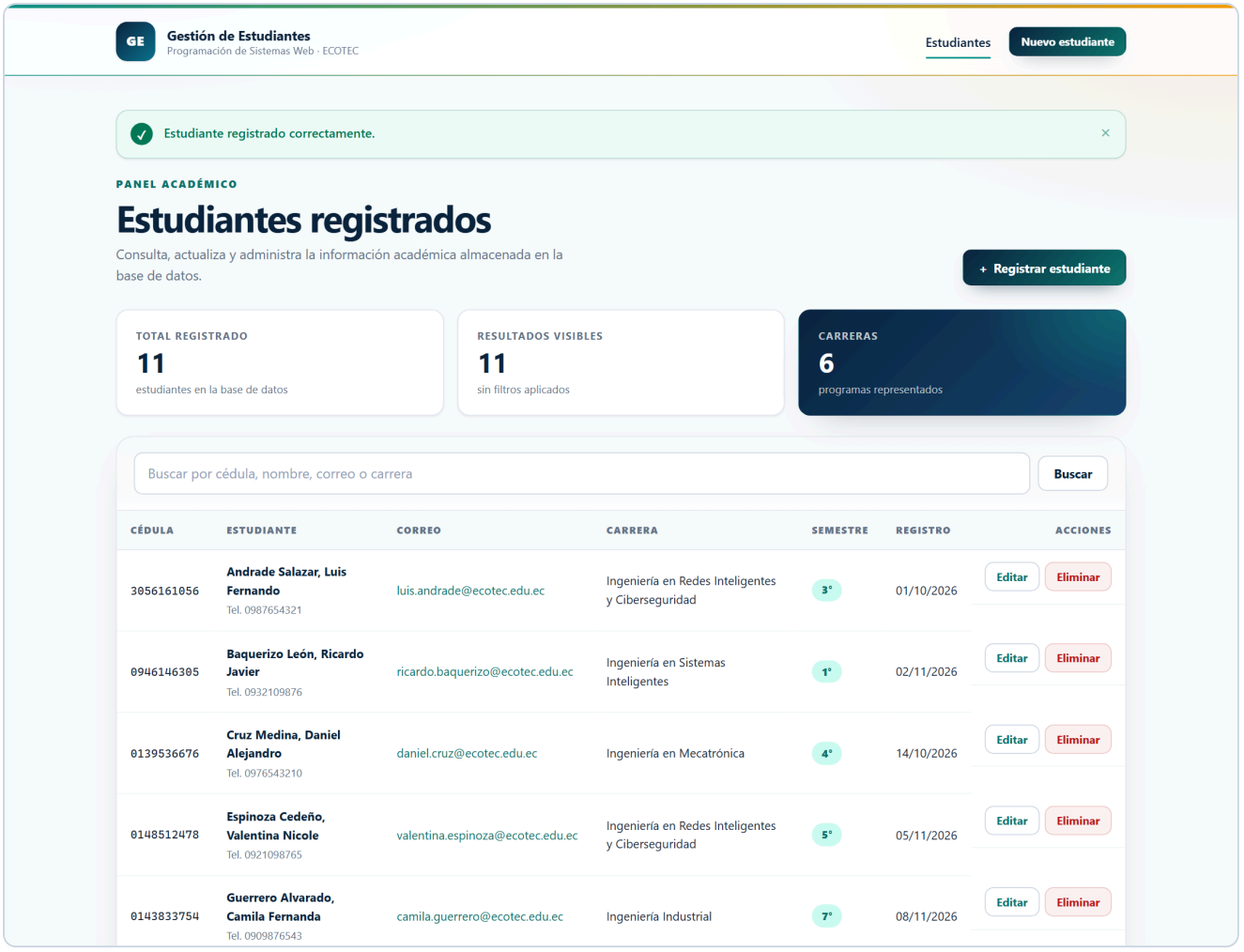


Figura 7. Tabla web con los registros insertados y mensaje de éxito. Evidencia obligatoria
Fuente: ejecución local del sistema, 2026.

Interpretación. El listado muestra once registros después de insertar el estudiante de prueba, el mensaje "Estudiante registrado correctamente" y las tarjetas de resumen (total, resultados visibles y carreras). Cada celda se genera con `htmlspecialchars()`, por lo que un valor malicioso se mostraría como texto. La tabla es responsiva: en pantallas pequeñas se transforma en tarjetas con etiquetas, como se aprecia en la Figura 11.

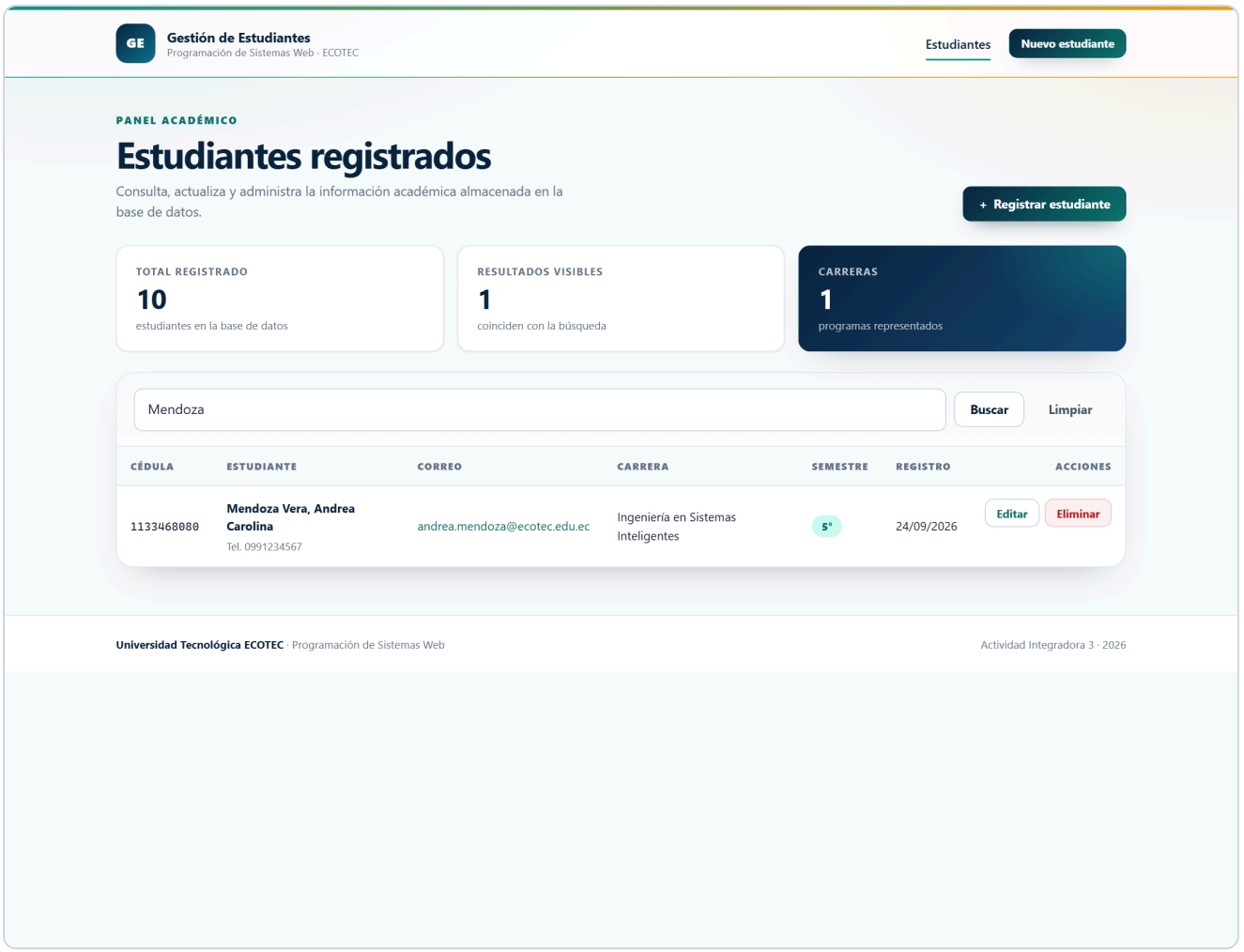


Figura 8. Búsqueda por apellido con consulta parametrizada.
Fuente: ejecución local del sistema, 2026.

Interpretación. La búsqueda por “Mendoza” devuelve la coincidencia y actualiza el contador de resultados. La consulta usa `LOCATE(?, columna)` con cinco parámetros posicionales, de modo que el término de búsqueda nunca se concatena en el SQL. El formulario de búsqueda conserva el término y ofrece la opción de limpiar el filtro.

4.6 Actualización y eliminación

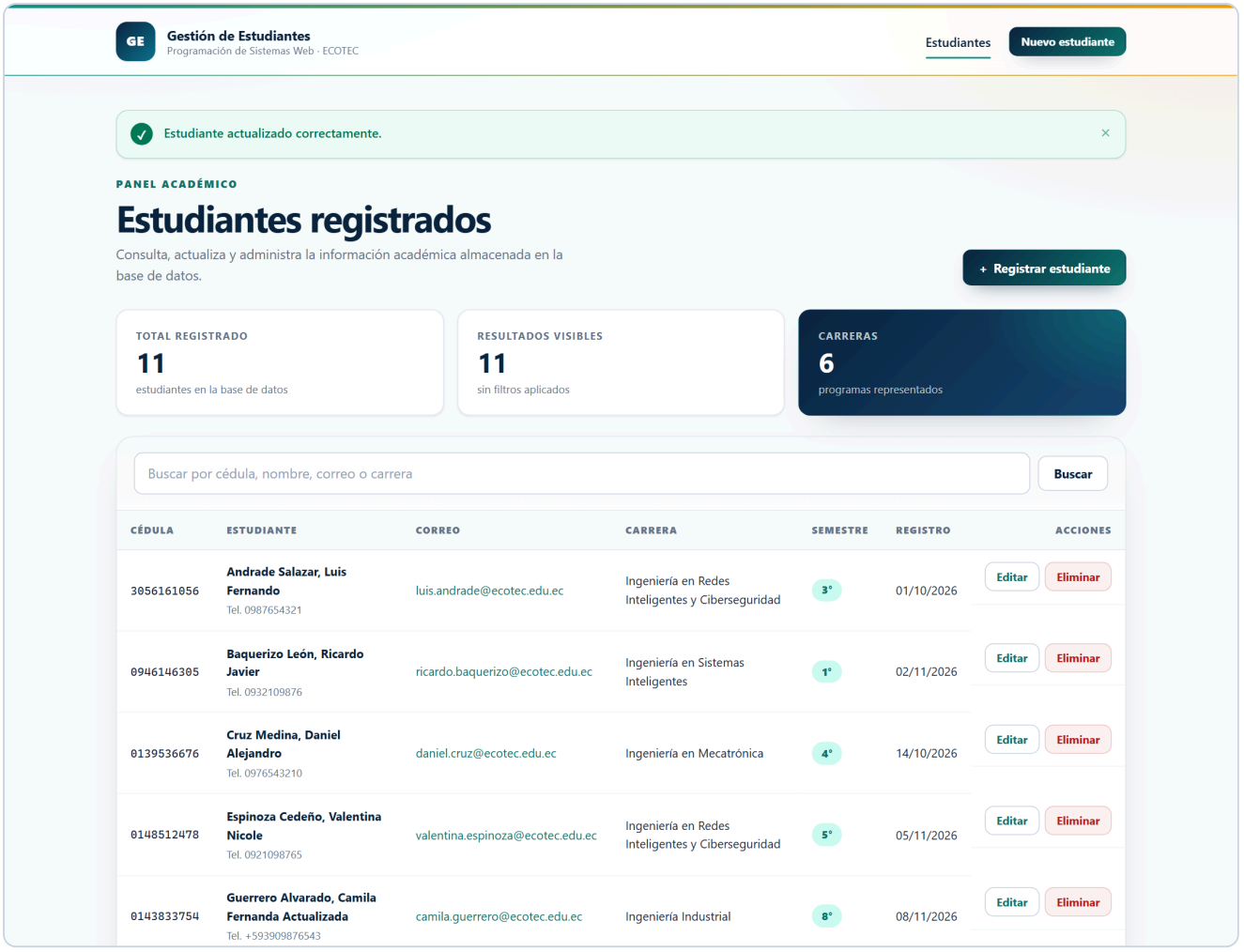


Figura 9. Actualización exitosa con patrón PRG y validación de unicidad.
Fuente: ejecución local del sistema, 2026.

Interpretación. El formulario de edición se carga con los datos actuales, permite modificarlos y, al guardar, redirige al listado con el mensaje "Estudiante actualizado correctamente". La validación de unicidad excluye el propio registro mediante `id <> :id`, por lo que el usuario puede conservar su cédula y correo sin falsos duplicados. El teléfono se normaliza antes de persistir.

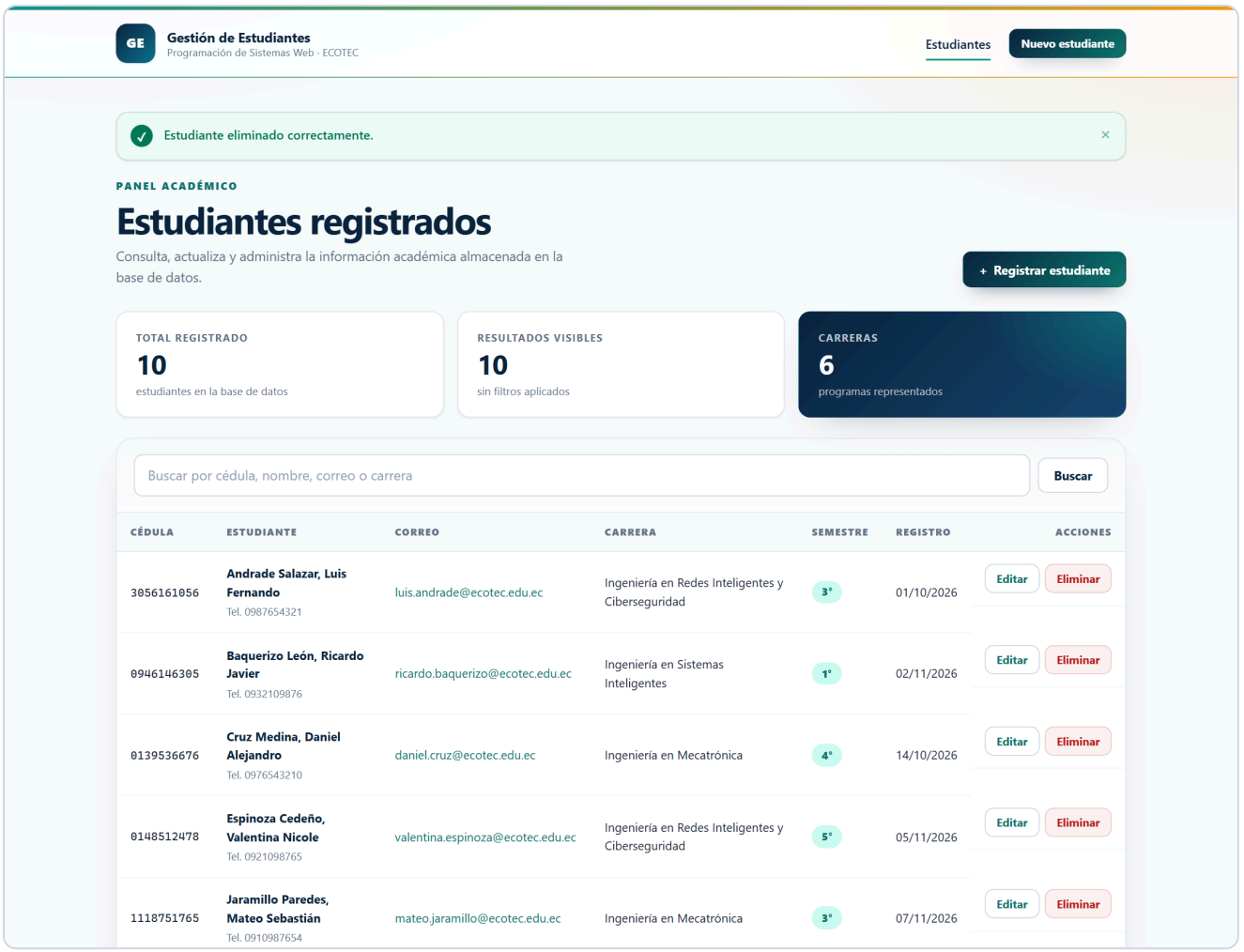


Figura 10. Eliminación con confirmación, CSRF y POST obligatorio.
Fuente: ejecución local del sistema, 2026.

Interpretación. La eliminación solicita confirmación, envía un POST con el token CSRF y el identificador validado, y responde con el mensaje “Estudiante eliminado correctamente”. El contador vuelve a diez registros y el estudiante de prueba ya no aparece. Una petición GET a la acción de eliminar sería rechazada con 405, lo que evita borrados accidentales por enlaces o precarga de recursos.

4.7 Diseño responsivo



Figura 11. Vista móvil del listado (390 × 844 px).
Fuente: ejecución local del sistema, 2026.

Interpretación. En un viewport móvil, la tabla se convierte en tarjetas con etiquetas (`data-label`), los botones se mantienen accesibles y la navegación se reorganiza. El diseño conserva la jerarquía visual y evita el desplazamiento horizontal. Esta captura respalda el criterio de interfaz responsiva y limpia.

4.8 Historial de commits

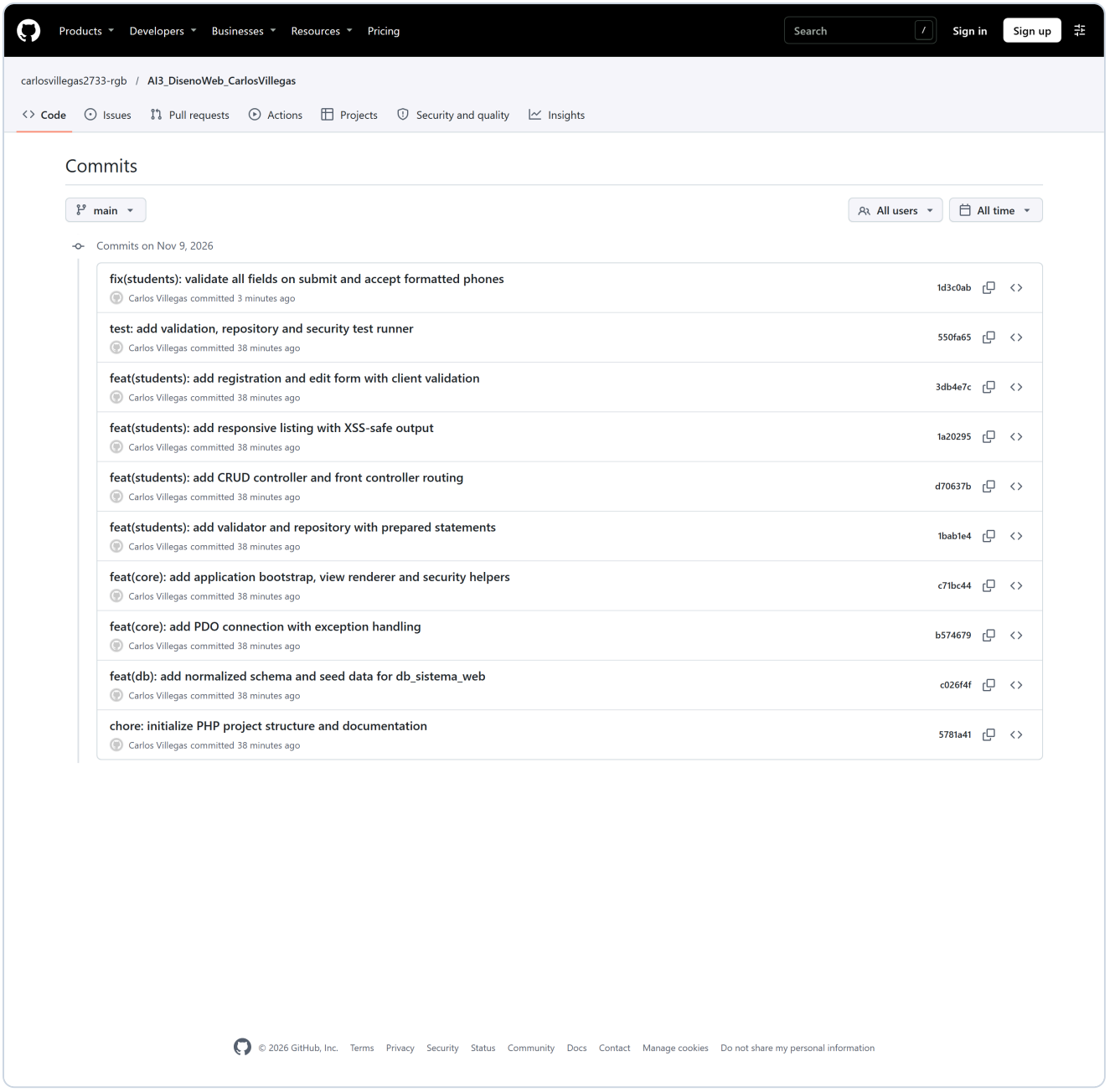


Figura 12. Historial de commits atómicos en GitHub. **Evidencia obligatoria**
Fuente: repositorio público, pestaña Commits, 2026.

Interpretación. El repositorio muestra diez commits de desarrollo con mensajes descriptivos y convención Conventional Commits: estructura y documentación, esquema de base de datos, conexión PDO, núcleo de la aplicación, modelo y validador, controlador, listado, formulario, pruebas y corrección de la validación. Cada commit representa una unidad de trabajo revisable y refleja un flujo real de control de versiones, no un único “initial commit”.

4.9 Verificación de datos en phpMyAdmin

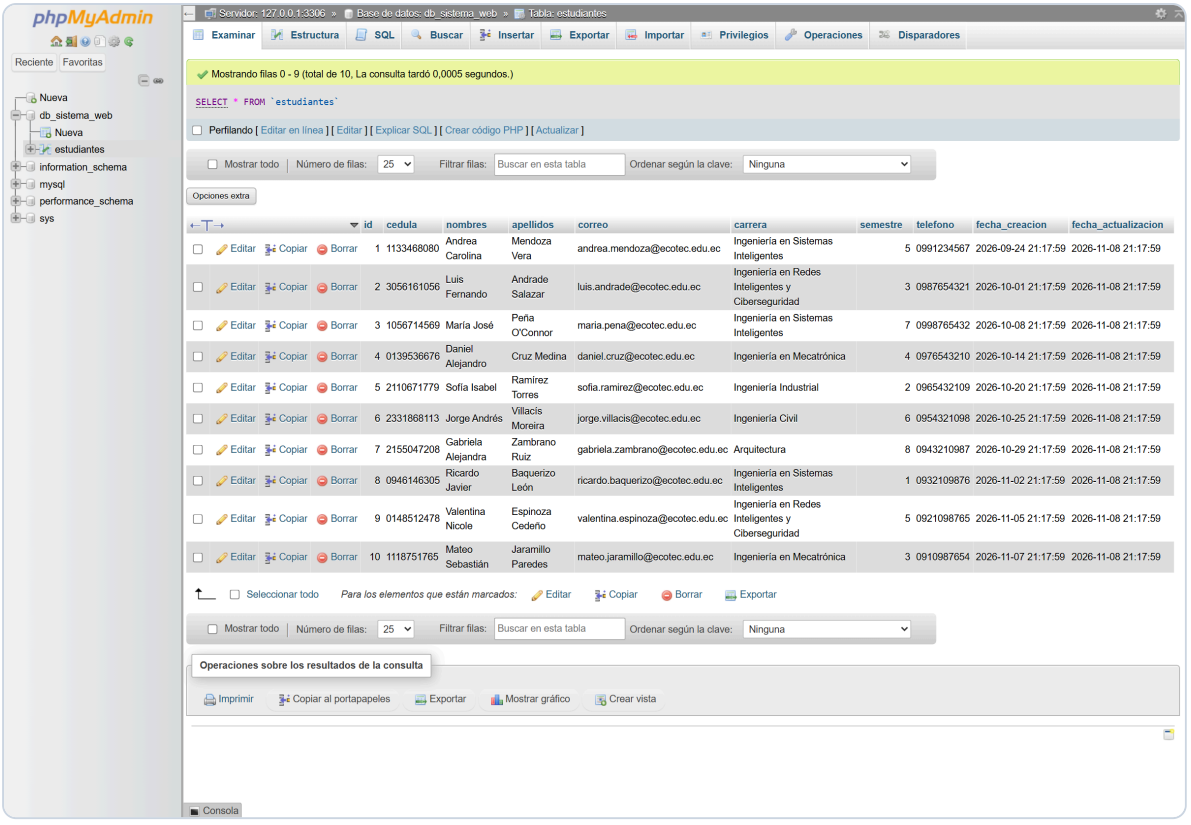


Figura 13. Datos persistidos en la tabla `estudiantes`.
Fuente: ejecución local del sistema, phpMyAdmin 5.2.3, 2026.

Interpretación. La vista de datos confirma que los diez registros de ejemplo se almacenaron con sus tipos correctos, timestamps de creación y actualización, y acentos preservados por `utf8mb4`. La integridad de los datos coincide con lo que muestra la aplicación y con las restricciones declaradas en el DDL.

5. Pruebas y resultados

La verificación combinó análisis estático, pruebas unitarias, integración HTTP y ejecución real en navegador. El objetivo fue comprobar no solo que el CRUD funciona, sino que las medidas de seguridad resisten entradas inválidas y ataques básicos.

5.1 Matriz de pruebas

Prueba	Alcance	Resultado
Sintaxis PHP	`php -l` sobre los 16 archivos del proyecto	16/16 correctos
Sintaxis JavaScript	`node --check public/assets/js/app.js`	Correcto
Pruebas unitarias	Normalización, reglas, cédula, escape HTML, repositorio y CRUD transaccional	34/34 aprobadas
Integración HTTP	Listado, creación, duplicados, edición, eliminación, CSRF, 400/404/405/419	30/30 aprobadas
XSS almacenado	Inserción de `<script>` y verificación de escape en el listado	Payload escapado; no se ejecuta
Flujo en navegador	Validación cliente/servidor, CRUD, búsqueda y vista móvil	Correcto

5.2 Resultados del runner de pruebas

php tests/run.php · resumen de ejecución

```
PASS  Normalizes the cédula to ten digits
PASS  Accepts a valid Ecuadorian cédula check digit
PASS  Rejects an invalid Ecuadorian cédula check digit
PASS  Rejects names with HTML characters
PASS  Escapes HTML output to prevent XSS
PASS  Detects an existing cédula
PASS  Searches students by surname
PASS  Creates a student with prepared statements
PASS  Persists the normalized phone
PASS  Persists the updated semester
PASS  Confirms the deleted student no longer exists

------------------------------------------------------------------------
Results: 34 passed, 0 failed, 34 total
All tests passed.
```

El runner utiliza una transacción con `rollBack()` para las pruebas de escritura, de modo que los registros de prueba no quedan en la base de datos. Las aserciones cubren los efectos observables: normalización de cédula, correo y teléfono, reglas de longitud y formato, dígito verificador, escape de HTML, búsqueda, creación, actualización y eliminación.

5.3 Pruebas de integración HTTP

Pruebas de integración · resumen

```
PASS  GET / returns 200
PASS  Listing renders the page title and seeded records
PASS  Server rejects an empty cédula, names, email and career
PASS  Server rejects a duplicate cédula and email
PASS  Valid create redirects with PRG and shows a success message
PASS  Edit form is populated with current data
PASS  Valid update redirects with PRG and updates the listing
PASS  Delete action removes the record and shows a success message
PASS  POST without CSRF token returns 419
PASS  GET on a mutating action returns 405
PASS  Unknown action returns 404
PASS  Invalid id returns 400

Integration results: 30 passed, 0 failed
```

Las pruebas de integración se ejecutaron contra el servidor PHP y la base de datos reales, usando una sesión con cookies y el token CSRF. Se verificaron los códigos de estado, los mensajes flash, la persistencia y la ausencia del registro después de eliminarlo. La prueba de XSS almacenado se ejecutó insertando una carga maliciosa mediante el repositorio y comprobando que el HTML de salida la contiene escapada y no como una etiqueta ejecutable.

> **Conclusión de las pruebas.** El sistema cumple el flujo funcional y las medidas de seguridad en el entorno local. Las pruebas son reproducibles con `php tests/run.php` y el script de integración incluido en la documentación del proyecto.

6. Enlaces, conclusiones y recomendaciones

6.1 Enlaces del proyecto

Repositorio público	`https://github.com/carlosvillegas2733-rgb/AI3_DisenoWeb_CarlosVillegas`
Informe PDF	`docs/AI3_Informe_CarlosVillegas.pdf`
Script de base de datos	`database/db_sistema_web.sql`
Evidencias	`docs/evidencias/`
Pruebas	`php tests/run.php`
Ejecución local	`php -S localhost:8000 -t public`

6.2 Conclusiones

PDO fue la elección adecuada. La portabilidad, el manejo de excepciones y los parámetros nombrados simplificaron el código y dejaron una puerta abierta a otros motores sin reescribir la capa de datos. La desactivación de la emulación de preparaciones garantiza que las consultas se ejecuten como sentencias preparadas reales del servidor.

La seguridad es una propiedad del conjunto. Las sentencias preparadas eliminan la inyección SQL, pero por sí solas no evitan XSS ni CSRF. La combinación de parametrización, escape de salidas, tokens CSRF, validación servidor, restricciones de base de datos y manejo controlado de errores construye una defensa en profundidad coherente con el nivel de exposición del módulo.

La arquitectura MVC facilita la evolución. Separar controlador, modelo, validador y vistas permitió probar la lógica sin renderizar HTML y mantener las vistas libres de SQL. El front controller centraliza las cabeceras y los errores, y el autoload PSR-4 mantiene la estructura ordenada.

El control de versiones documenta el proceso. Los commits atómicos con mensajes descriptivos muestran la evolución real del proyecto: desde la estructura y el esquema hasta la conexión, el CRUD, las pruebas y la corrección de la validación. El repositorio público y el README permiten que otra persona reproduzca el entorno.

6.3 Recomendaciones para producción

- Servir la aplicación bajo HTTPS y mantener las cookies `Secure` además de `HttpOnly` y `SameSite`.
- Crear un usuario de base de datos con privilegios mínimos (SELECT, INSERT, UPDATE, DELETE sobre `db_sistema_web`) en lugar de usar `root`.
- Gestionar credenciales con variables de entorno o un gestor de secretos; nunca versionar archivos `.env`.
- Activar `display_errors=Off` y `log_errors=On` en producción; centralizar los logs y alertar sobre errores 500.
- Agregar autenticación, roles y auditoría si el sistema se extiende a varios usuarios.
- Incorporar paginación y filtros combinados cuando el volumen de registros crezca.

- Automatizar las pruebas con PHPUnit y un flujo de integración continua (GitHub Actions) con análisis estático (PHPStan).
- Programar copias de seguridad y probar la restauración de la base de datos.

6.4 Referencias

- PHP Group. *PHP Data Objects (PDO)*. https://www.php.net/manual/en/book.pdo.php
- PHP Group. *PDO::prepare*. https://www.php.net/manual/en/pdo.prepare.php
- PHP Group. *PDO::setAttribute — ATTR_EMULATE_PREPARES*. https://www.php.net/manual/en/pdo.setattribute.php
- PHP Group. *htmlspecialchars*. https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Group. *filter_var*. https://www.php.net/manual/en/function.filter-var.php
- OWASP. *SQL Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP. *Cross Site Scripting Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP. *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- MariaDB Foundation. *MariaDB Documentation*. https://mariadb.com/kb/en/
- Oracle. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/
- PHP-FIG. *PSR-12: Extended Coding Style*. https://www.php-fig.org/psr/psr-12/
- Conventional Commits. *Specification*. https://www.conventionalcommits.org/

Anexo A. Script de creación de la base de datos

El archivo `database/db_sistema_web.sql` contiene el DDL completo y diez registros de ejemplo con cédulas ecuatorianas válidas. El siguiente extracto muestra la definición de la base y de la tabla principal:

database/db_sistema_web.sql · DDL

```
CREATE DATABASE IF NOT EXISTS db_sistema_web
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE db_sistema_web;

CREATE TABLE estudiantes (
    id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    cedula CHAR(10) NOT NULL,
    nombres VARCHAR(80) NOT NULL,
    apellidos VARCHAR(80) NOT NULL,
    correo VARCHAR(120) NOT NULL,
    carrera VARCHAR(80) NOT NULL,
    semestre TINYINT UNSIGNED NOT NULL,
    telefono VARCHAR(16) NOT NULL,
    fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_estudiantes_cedula (cedula),
    UNIQUE KEY uq_estudiantes_correo (correo),
    KEY idx_estudiantes_apellidos (apellidos),
    KEY idx_estudiantes_carrera (carrera),
    CONSTRAINT chk_estudiantes_semestre CHECK (semestre BETWEEN 1 AND 8)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4
  COLLATE = utf8mb4_unicode_ci;
```

Tipos y restricciones. `INT UNSIGNED AUTO_INCREMENT` garantiza un identificador único y eficiente; `CHAR(10)` es adecuado para una cédula de longitud fija; `VARCHAR` se usa para textos variables; `TINYINT UNSIGNED` almacena el semestre en un byte; `UNIQUE` protege la unicidad de cédula y correo incluso ante errores de la capa superior; `CHECK` restringe el semestre; y los `TIMESTAMP` registran la auditoría de creación y actualización. Los índices de apellidos y carrera aceleran las búsquedas y el ordenamiento.

database/db_sistema_web.sql · ejemplo de datos

```
INSERT INTO estudiantes
    (cedula, nombres, apellidos, correo, carrera, semestre, telefono, fecha_creacion)
VALUES
    ('1133468080', 'Andrea Carolina', 'Mendoza Vera',
     'andrea.mendoza@ecotec.edu.ec',
     'Ingeniería en Sistemas Inteligentes', 5, '0991234567',
     DATE_SUB(NOW(), INTERVAL 45 DAY)),
    ('3056161056', 'Luis Fernando', 'Andrade Salazar',
     'luis.andrade@ecotec.edu.ec',
     'Ingeniería en Redes Inteligentes y Ciberseguridad', 3, '0987654321',
     DATE_SUB(NOW(), INTERVAL 38 DAY));
```

Anexo B. Guía de despliegue local

Requisitos

- PHP 8.1 o superior con `pdo_mysql`, `mbstring` y `openssl`.
- MySQL 8.x o MariaDB 10.4+.
- XAMPP, WAMP, Laragon o el servidor integrado de PHP.
- Git para clonar el repositorio (opcional).

Pasos

1. Clonar el repositorio dentro de `htdocs`:

```
git clone https://github.com/carlosvillegas2733-rgb/AI3_DisenoWeb_CarlosVillegas.git
```

2. Iniciar Apache y MySQL desde el panel de XAMPP.
3. Importar `database/db_sistema_web.sql` desde phpMyAdmin (pestaña Importar).
4. Revisar `app/Config/conexion.php`. Los valores por defecto son `127.0.0.1`, puerto `3306`, base `db_sistema_web`, usuario `root` y contraseña vacía.
5. Abrir `http://localhost/AI3_DisenoWeb_CarlosVillegas/public/` en el navegador.

Servidor integrado de PHP

Ejecución sin Apache

```
php -S localhost:8000 -t public
```

Luego abrir `http://localhost:8000`.

Variables de entorno opcionales

La conexión acepta `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` y `DB_PASSWORD`. Si no se definen, se usan los valores locales indicados. Esto permite desplegar en otro entorno sin modificar el código.

Pruebas

Verificación del proyecto

```
php tests/run.php

# Resultado esperado
Results: 34 passed, 0 failed, 34 total
All tests passed.
```

> **Repositorio público:** `https://github.com/carlosvillegas2733-rgb/AI3_DisenoWeb_CarlosVillegas`